Unit 5 · Lists and functions

CONCEPT & REASONING

Keep related values in a list and organise repeated reasoning into reusable functions. This unit builds on loops and decisions.

EXAMPLE / TAKEAWAY

From five separate amounts → one list → small functions → a clear expenditure report.

Learning goals

CONCEPT & REASONING

Create and change lists, access elements safely, traverse a collection, define functions and use returned results in calculations.

EXAMPLE / TAKEAWAY

Final task: five validated expenses stored in a list and analysed by three custom functions.

Why store several values?

CONCEPT & REASONING

A running total preserves only the sum. A list preserves the individual observations so they can be inspected, corrected and summarised again.

PYTHON EXAMPLE

```
expenses = [2000, 3000, 0, 4000, 1000]
print(expenses)  # all five values
```

A list is an ordered collection

CONCEPT & REASONING

Lists use square brackets and comma-separated elements. A numeric list may contain repeated values and zeros.

PYTHON EXAMPLE

```
numbers = [10, 20, 10]
empty = []
print(type(numbers))  # <class 'list'>
```

One name, several elements

CONCEPT & REASONING

Each position has an index. Python starts at 0, not 1. For five items, the valid positive indices are 0 through 4.

EXAMPLE / TAKEAWAY

values: [2000] [3000] [0] [4000] [1000]

indices: 0 1 2 3 4

Read the first element

CONCEPT & REASONING

Use list[index] to retrieve a value. The first element is found at index 0.

PYTHON EXAMPLE

```
expenses = [2000, 3000, 0]
print(expenses[0])  # 2000
print(expenses[1])  # 3000
```

Read the last element

CONCEPT & REASONING

Negative indices count from the end. -1 is the final value, and -2 is the previous one.

PYTHON EXAMPLE

```
expenses = [2000, 3000, 0]
print(expenses[-1])  # 0
print(expenses[-2])  # 3000
```

Avoid IndexError

CONCEPT & REASONING

An index must refer to a position that exists. For a three-element list, index 3 is out of range.

PYTHON EXAMPLE

```
expenses = [2000, 3000, 0]
print(expenses[2])  # valid: 0
# print(expenses[3]) would fail
```

Count elements using len()

CONCEPT & REASONING

len(list) returns the number of elements, which is not the last index. An empty list has length zero.

PYTHON EXAMPLE

```
expenses = [2000, 3000, 0]
print(len(expenses))  # 3
print(len([]))        # 0
```

Check if a value occurs

CONCEPT & REASONING

The in operator checks membership. A Boolean expression can be used inside if to report a zero-spend day.

PYTHON EXAMPLE

```
expenses = [2000, 0, 3000]
print(0 in expenses)       # True
print(500 not in expenses) # True
```

Change an element

CONCEPT & REASONING

Assign to an existing index. This changes the stored value at that position and preserves the list length.

PYTHON EXAMPLE

```
expenses = [2000, 3000, 0]
expenses[1] = 2500
print(expenses)  # [2000, 2500, 0]
```

Append a new item

CONCEPT & REASONING

append() adds exactly one new element at the end. A loop can call append once for every accepted input.

PYTHON EXAMPLE

```
expenses = []
expenses.append(2000)
expenses.append(3000)
print(expenses)  # [2000, 3000]
```

append() versus extend()

CONCEPT & REASONING

append(item) adds one element. extend(collection) appends the separate elements of that collection.

PYTHON EXAMPLE

```
a = [1]
a.append([2, 3])  # [1, [2, 3]]
b = [1]
b.extend([2, 3])  # [1, 2, 3]
```

Insert at a position

CONCEPT & REASONING

insert(index, value) adds an element before the requested position; existing items shift to the right.

PYTHON EXAMPLE

```
expenses = [2000, 4000]
expenses.insert(1, 3000)
print(expenses)  # [2000, 3000, 4000]
```

Remove the last element

CONCEPT & REASONING

pop() removes and returns the final element. Use only when the list is non-empty.

PYTHON EXAMPLE

```
expenses = [2000, 3000, 0]
last = expenses.pop()
print(last)      # 0
print(expenses)  # [2000, 3000]
```

Remove by value

CONCEPT & REASONING

remove(value) removes the first matching item, not the item at that numeric index. The value must occur.

PYTHON EXAMPLE

```
expenses = [2000, 3000, 2000]
expenses.remove(2000)
print(expenses)  # [3000, 2000]
```

Slices select part of a list

CONCEPT & REASONING

Like range(), slicing includes the start and excludes the stop. The result is a new list of the selected elements.

PYTHON EXAMPLE

```
values = [10, 20, 30, 40, 50]
print(values[1:4]) # [20, 30, 40]
print(values[:2])  # [10, 20]
```

Loop over a list directly

CONCEPT & REASONING

Use for item in items when you need the values themselves. The body executes once for each element.

PYTHON EXAMPLE

```
expenses = [2000, 3000, 0]
for amount in expenses:
    print(amount)  # 3 lines
```

Display each day with its index

CONCEPT & REASONING

When a day number matters, enumerate can give both the display position and the amount. start=1 is a human-friendly label.

PYTHON EXAMPLE

```
expenses = [2000, 3000, 0]
for day, amount in enumerate(expenses,
1):
    print(day, amount)
```

Calculate a total with a loop

CONCEPT & REASONING

Start an accumulator before the loop. Add each element to the running total, then print the final result once.

PYTHON EXAMPLE

```
total = 0
for amount in [2000, 3000, 0]:
    total += amount
print(total)  # 5000
```

Summarise with built-in functions

CONCEPT & REASONING

sum(), min() and max() are convenient for non-empty numeric lists. Check special cases before using min or max.

PYTHON EXAMPLE

```
x = [12, 15, 10, 18]
print(sum(x))  # 55
print(min(x))  # 10
print(max(x))  # 18
```

Calculate the average safely

CONCEPT & REASONING

Average equals total divided by the number of observations. An empty list has no ordinary numerical average.

PYTHON EXAMPLE

```
values = [1000, 2000, 3000]
average = sum(values) / len(values)
print(average)  # 2000.0
```

Collect data with append()

CONCEPT & REASONING

Start with an empty list. After a valid amount has been entered, append it. Repeat five times.

PYTHON EXAMPLE

```
expenses = []
for day in range(1, 6):
    amount = float(input(f"Day {day}: "))
    expenses.append(amount)
```

Validate before appending

CONCEPT & REASONING

Use a while loop for a negative numeric amount. Append only when the accepted value is non-negative.

PYTHON EXAMPLE

```
while amount < 0:
    amount = float(input("Retry: "))
expenses.append(amount)
```

Trace the growing list

CONCEPT & REASONING

The list changes after each successful append. A negative attempt does not create a new day.

EXAMPLE / TAKEAWAY

Day 1: 2000 → [2000]

Day 2: -500 rejected; 0 → [2000, 0]

Day 3: 1000 → [2000, 0, 1000]

What is a function?

CONCEPT & REASONING

A function names a reusable set of statements for one task. It is defined with def and executed by calling its name.

PYTHON EXAMPLE

```
def greet():
    print("Welcome")

greet()  # displays Welcome
```

Definition versus call

CONCEPT & REASONING

Python reads the definition but does not run its body until the function is called. Statements after the definition can call it many times.

PYTHON EXAMPLE

```
def announce():
    print("Recorded")

announce()
announce()  # displays twice
```

Parameters are named inputs

CONCEPT & REASONING

Parameters are names inside def (...). Arguments are the values passed during the call.

PYTHON EXAMPLE

```
def show_cost(amount):
    print(f"TZS {amount:.2f}")
show_cost(2500)  # 2500 is argument
```

Two parameters

CONCEPT & REASONING

A function can receive multiple values. The caller should supply the values in the intended order, or use keyword arguments.

PYTHON EXAMPLE

```
def show_day(day, amount):
    print(day, amount)
show_day(2, 3500)
show_day(amount=3500, day=2)
```

Return gives a result to the caller

CONCEPT & REASONING

Use return when another part of the programme needs the computed value. It ends that function call.

PYTHON EXAMPLE

```
def double_value(n):
    return 2 * n
result = double_value(1500)
print(result + 500)  # 3500
```

print() is not return

CONCEPT & REASONING

Printing displays a value but does not make it a reusable return result. A function without return produces None.

PYTHON EXAMPLE

```
def display(n):
    print(n * 2)
value = display(5)  # prints 10
print(value)         # None
```

Local variables belong to a call

CONCEPT & REASONING

Local variables created inside a function cannot normally be read by name from the calling code. Return what the caller needs.

PYTHON EXAMPLE

```
def price_with_tax(price):
    tax = price * 0.18
    return price + tax
print(price_with_tax(1000)) # 1180.0
```

Write a total function

CONCEPT & REASONING

The function accepts any list of numeric values, accumulates their sum and returns it. It does not print or change the list.

PYTHON EXAMPLE

```
def calculate_total(values):
    total = 0.0
    for value in values:
        total += value
    return total
```

Call calculate_total()

CONCEPT & REASONING

A single function works with different lists. The caller receives a numeric result that can be formatted or used in comparisons.

PYTHON EXAMPLE

```
print(calculate_total([2000,3000,0])) #
5000.0
print(calculate_total([])) # 0.0
```

Write a zero-day counter

CONCEPT & REASONING

The function checks every value and counts entries equal to zero. This is different from counting negative values.

PYTHON EXAMPLE

```
def count_zero_days(values):
    count = 0
    for value in values:
        if value == 0:
            count += 1
    return count
```

Use the zero-day function

CONCEPT & REASONING

For [0, 1000, 0, 3000, 0], exactly three elements equal zero. The count returned is 3.

PYTHON EXAMPLE

```
expenses = [0, 1000, 0, 3000, 0]
print(count_zero_days(expenses))  # 3
```

Return a budget message

CONCEPT & REASONING

A decision function can return a text label. The main programme handles displaying that message.

PYTHON EXAMPLE

```
def budget_message(total, allowance):
    if total > allowance:
        return "Over budget"
    if total == allowance:
        return "Exactly on budget"
    return "Within budget"
```

Trace the call from input to return

CONCEPT & REASONING

Pass a list to calculate_total, initialise local total, add each element in order, return it, and store the result in the caller.

EXAMPLE / TAKEAWAY

[2000,3000,0]

local: 0 → 2000 → 5000 → 5000

return 5000 → report displays 5000

Functions can be tested separately

CONCEPT & REASONING

A good test checks a small function before building the entire programme. Verify normal, zero and boundary examples.

EXAMPLE / TAKEAWAY

calculate_total([]) → 0.0

count_zero_days([0, 10, 0]) → 2

budget_message(10, 10) → Exactly on budget

Build the main programme

CONCEPT & REASONING

Keep input in the main script; use helper functions for calculations. This structure is easier to read and debug.

PYTHON EXAMPLE

```
allowance = 20000.0
expenses = [2000, 3000, 0, 4000, 1000]
total = calculate_total(expenses)
zeros = count_zero_days(expenses)
```

Calculate the daily average

CONCEPT & REASONING

When five values are collected, divide their total by len(expenses). Display money with units and two decimal places.

PYTHON EXAMPLE

```
average = total / len(expenses)
print(f"Total: TZS {total:.2f}")
print(f"Average: TZS {average:.2f}")
```

Display the final budget status

CONCEPT & REASONING

The status comes from one named function; it is not duplicated in several parts of the programme.

PYTHON EXAMPLE

```
message = budget_message(total,
allowance)
print("Zero-spend days:", zeros)
print("Status:", message)
```

Complete expected example

CONCEPT & REASONING

An allowance of 20000 and expenses [2000,3000,0,4000,1000] produce a simple, verifiable report.

EXAMPLE / TAKEAWAY

Total: TZS 10000.00

Average: TZS 2000.00

Zero-spend days: 1

Status: Within budget

Test boundary behaviour

CONCEPT & REASONING

Include all-zero spending, exact allowance match, overspending and a rejected negative entry. Check the stored list length.

EXAMPLE / TAKEAWAY

All zero: total 0, zeros 5

Exact 15000: status Exactly on budget

Negative input: retry same day

Avoid common list mistakes

CONCEPT & REASONING

Accessing index 5 in a five-item list fails. Appending before validation stores bad data. Assigning b = a does not copy a list.

EXAMPLE / TAKEAWAY

Right: expenses[4] for final item

Right: validate → append

Copy when needed: other = expenses.copy()

Avoid common function mistakes

CONCEPT & REASONING

A return statement must provide the result to the caller. Initialise the local accumulator once, outside its loop.

PYTHON EXAMPLE

```
def bad(values):
    for value in values:
        total = 0  # resets each pass
# Correct: total = 0 before the loop
```

Python collections and their purposes

CONCEPT & REASONING

A list is an ordered mutable collection; a tuple is ordered but its element positions cannot be reassigned; a dictionary associates keys with values; a set represents unique elements without a guaranteed positional order. A Python list is not a fixed-size typed array; its elements may have different types. Lists remain the main collection for this course.

EXAMPLE & RESULT

```
marks = [70, 85, 90]
point = (10, 20)
student = {"name": "Amina", "score": 85}
unique = {1, 2, 2, 3}
print(student["score"], len(unique))
```

85 3. Dictionary keys name values; the set removes duplicates.

List references and independent copies

CONCEPT & REASONING

Assigning one list name to another does not copy the list. Both names refer to the same mutable object; changing an element through either name affects that object. list.copy() makes a shallow copy, sufficient for a list of numbers. Nested lists can still share inner objects.

EXAMPLE & RESULT

```
marks = [70, 80, 90]
other = marks
other[0] = 100
print(marks)
copy_marks = marks.copy()
copy_marks[0] = 60
print(marks, copy_marks)
```

[100, 80, 90]; then [100, 80, 90] and [60, 80, 90].

Functions, side effects and mutation

CONCEPT & REASONING

A function can return a new value without changing its input, or it can deliberately mutate a list passed as an argument. Side effects are changes outside the returned result. A clear function contract describes whether the input changes.

EXAMPLE & RESULT

```
def increased(values):
    result = []
    for v in values:
        result.append(v + 5)
    return result

marks = [70, 80]
print(increased(marks))
print(marks)
```

[75, 85] and [70, 80]. The loop creates a new list with append(); the original remains unchanged.

Modular design: separate calculation from output

CONCEPT & REASONING

A module is a file containing Python definitions or statements. Modular design divides responsibilities: input collection, computation and presentation. A function that returns a result can be reused in another programme and tested without reading keyboard input or printing.

EXAMPLE & RESULT

```
def mean_mark(marks):
    if len(marks) == 0:
        return None
    return sum(marks) / len(marks)

result = mean_mark([70, 80, 90])
print("Mean:", result)
```

Mean: 80.0. An empty list returns None instead of dividing by zero.

Function documentation and imports

CONCEPT & REASONING

A docstring is a string at the start of a function body that describes its purpose, parameters, result and assumptions. Python provides built-in functions such as len(), while modules provide imported functions such as math.sqrt(). A module import makes its definitions available; no extra installation is needed for math.

EXAMPLE & RESULT

```
import math

def circle_area(radius):
    """Return area for a non-negative radius."""
    return math.pi * radius ** 2

print(round(circle_area(2), 2))
```

12.57. The docstring states the non-negative-radius assumption; the caller is responsible for supplying a valid radius.

A data-analysis function with an explicit contract

CONCEPT & REASONING

A reusable function has a defined input, output and valid input range. For numeric measurements, an empty collection has no arithmetic mean. Returning None makes this case explicit rather than reporting zero, which could be a valid mean.

EXAMPLE & RESULT

```
def mean_value(values):
    """Return mean of numbers, or None if empty."""
    if not values:
        return None
    return sum(values) / len(values)

print(mean_value([0, 2, 4]))
print(mean_value([]))
```

2.0; None. The examples test both ordinary and empty inputs.

Activity U5: list-based analyser

CONCEPT & REASONING

Record five non-negative expenses in one list. Write separate functions for total, zero count and budget status. Show a formatted report.

EXAMPLE / TAKEAWAY

Deliverable: StudentID_U5.py + a test-notes document. Five specified scenarios. 20 raw marks.

What to submit

CONCEPT & REASONING

Explain your input and output design, the list and each function, and expected-versus-actual test results. Declare any assistance used.

EXAMPLE / TAKEAWAY

Upload using the course Google Form. Keep a copy of your runnable programme and the form confirmation.

Coming next: testing and debugging

CONCEPT & REASONING

Unit 6 deepens your ability to detect faults, isolate causes and demonstrate that code works as expected.

EXAMPLE / TAKEAWAY

You are ready if you can manipulate a list, follow a function call and explain what return sends back.